Python Engineering - Classes & Objects Q&A;

Donde poner la clase y los metodos, que es self y donde vive una variable calculada

1. En que modulo pongo la clase?

Para EX21, coloca la definicion completa de `class VerticalState` en `vertical_state.py`.

```
ex21_vertical_state_class/
|-- vertical_state.py
`-- main.py
```

`vertical_state.py` define **que es** un VerticalState. `main.py` crea objetos y los utiliza.

2. Donde pongo los metodos?

Los metodos que pertenecen a VerticalState se escriben **dentro de la clase**, por tanto tambien viven en `vertical_state.py`.

```
# vertical_state.py

import numpy as np

class VerticalState:

    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def specific_energy(self):
        ...

    def state_vector(self):
        ...

    def euler_step(self, dt):
        ...
```

La indentacion importa: si specific_energy() esta indentado dentro de class VerticalState, es un metodo de esa clase.

3. Entonces, un metodo es como una funcion?

Si. La idea mas sencilla es: **un metodo es una funcion definida dentro de una clase**. La diferencia importante es que normalmente recibe el objeto actual mediante self y puede usar sus atributos.

```
# normal function
def specific_energy(h, v, g):
    return g*h + 0.5*v**2

# method
def specific_energy(self):
    return self.g*self.h + 0.5*self.v**2
```

La funcion normal necesita que le pases h, v y g. El metodo ya puede encontrarlos dentro del propio objeto.

4. Podriamos calcular e y devolver e?

Si. De hecho es perfectamente correcto y muy claro:

```
def specific_energy(self):
    e = self.g * self.h + 0.5 * self.v**2
    return e
```

Aqui e es una **variable local del metodo**. Existe mientras se ejecuta specific_energy(). Al terminar, no queda automaticamente almacenada como atributo del objeto.

5. Entonces, donde se almacena e?

Si hacemos:

```
earth_energy = earth_state.specific_energy()
```

el valor retornado queda guardado en `earth_energy`, que es una variable del codigo que hizo la llamada. No queda dentro de earth_state a menos que lo guardemos expresamente como atributo.

```
def specific_energy(self):
    self.e = self.g * self.h + 0.5 * self.v**2
    return self.e
```

Esto si crearia `self.e` dentro del objeto. Pero para EX21 **no lo recomiendo**: la energia depende de h y v, y si luego euler_step() cambia h y v, self.e podria quedarse desactualizada.

6. Que conviene hacer con una magnitud derivada como energia?

Para una magnitud que se puede recalcular facilmente a partir del estado actual, es mas seguro calcularla cuando se necesita:

```
def specific_energy(self):
    e = self.g * self.h + 0.5 * self.v**2
    return e
```

Asi la energia siempre corresponde a los valores actuales de self.h, self.v y self.g.

7. Por que usamos self.h en vez de h?

Porque dentro de specific_energy() queremos usar **la altura almacenada en ese objeto**. En ese metodo no existe una variable local h a menos que la pasemos explicitamente.

```
def specific_energy(self):
    # h does not exist here by itself
    # self.h is stored in the object
    return self.g * self.h + 0.5 * self.v**2
```

Si earth_state.h=100, self.h vale 100 cuando el metodo lo ejecuta earth_state. Si moon_state.h=250, self.h vale 250 cuando lo ejecuta moon_state.

8. Por que no pasar h, v y g otra vez al metodo?

Podriamos escribir esto:

```
def specific_energy(self, h, v, g):
    return g*h + 0.5*v**2
```

pero perderiamos parte de la utilidad del objeto: ya hemos guardado h, v y g dentro de el. Obligarnos a pasarlos otra vez duplicaria informacion y podria crear inconsistencias.

```
earth_state.h = 100.0

# Bad/confusing possibility:
earth_state.specific_energy(h=500.0, v=20.0, g=9.81)
# Which height represents the object's state?
```

El metodo debe usar self.h cuando la informacion pertenece al estado del objeto.

9. Por que self aparece en la definicion del metodo?

Porque Python necesita recibir una referencia al objeto concreto. Cuando escribimos:

```
earth_state.specific_energy()
```

Python hace conceptualmente algo equivalente a:

```
VerticalState.specific_energy(earth_state)
```

El objeto earth_state entra como primer argumento y dentro del metodo recibe el nombre self.

10. Por que no escribimos self al llamar?

Porque al llamar el metodo mediante un objeto, Python lo pasa automaticamente.

```
# Definition:
def euler_step(self, dt):
    ...

# Call:
earth_state.euler_step(0.1)

# Conceptually:
VerticalState.euler_step(earth_state, 0.1)
```

11. Que cosas llevan self. y cuales no?

Elemento	Ejemplo	Por que
Atributo persistente del objeto	self.h	Pertenece al estado del objeto
Otro atributo persistente	self.g	Cada objeto puede tener un valor diferente
Parametro temporal del metodo	dt	Solo se necesita durante esa llamada
Variable local calculada	e	Solo sirve durante el calculo si no queremos almacenarla

12. Estructura completa recomendada para EX21

```
# vertical_state.py
import numpy as np

class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g

    def specific_energy(self):
        e = self.g*self.h + 0.5*self.v**2
        return e

    def state_vector(self):
        return np.array([self.h, self.v, self.t])

    def is_ascending(self):
        return self.v > 0.0

    def euler_step(self, dt):
        self.h = self.h + self.v*dt
        self.v = self.v - self.g*dt
        self.t = self.t + dt

# main.py
from vertical_state import VerticalState

def main():
    earth_state = VerticalState(100.0, 20.0)
    energy = earth_state.specific_energy()
    earth_state.euler_step(0.1)
    print(energy)
    print(earth_state.state_vector())

if __name__ == '__main__':
    main()
```

Resumen final

vertical_state.py contiene la clase y sus metodos.
main.py crea y utiliza objetos.
Un **metodo** es una funcion asociada a una clase.

self representa el objeto actual.
self.h es estado almacenado; **h** puede ser solo un parametro temporal.
Una variable local como **e** puede devolverse sin almacenarla en el objeto.
Si escribimos **self.e**, entonces si se convierte en atributo persistente, pero puede quedar desactualizado si cambia el estado.